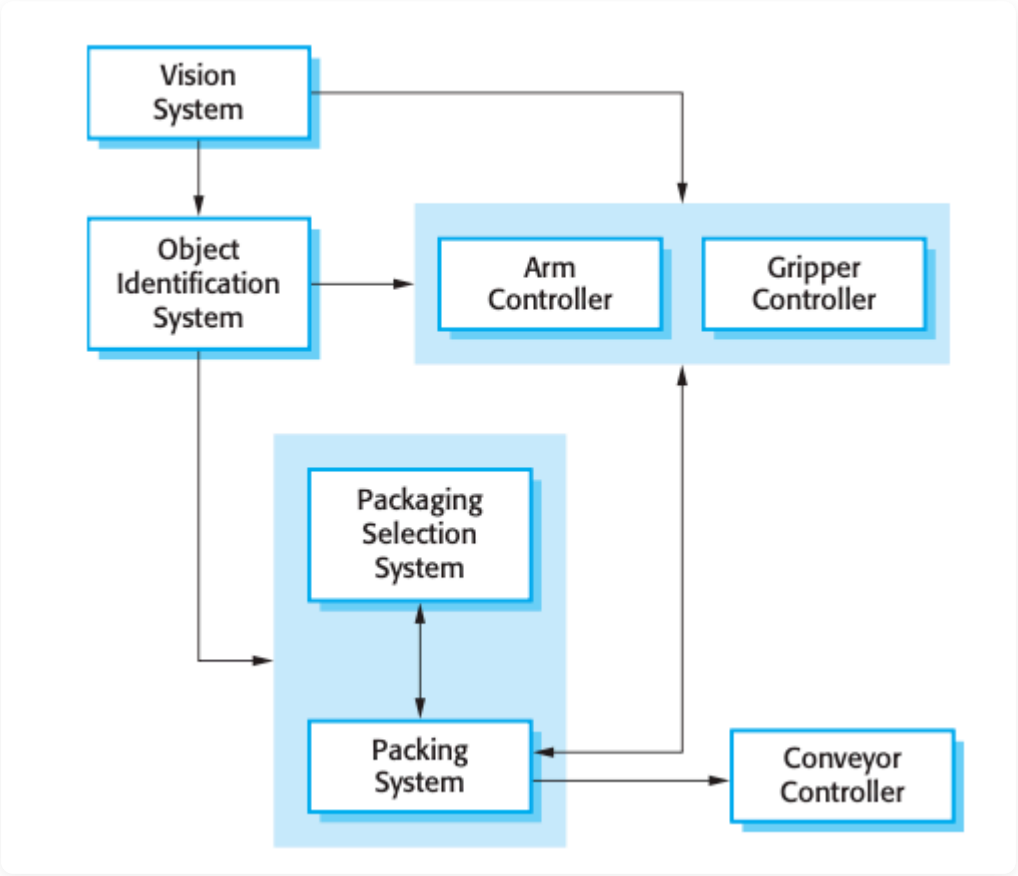
Architecture design

:::success 建築師為什麼重要？ :::

Basic concept

- **Architectural design** is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- Architectural design is the critical link between **design** and **requirements engineering**, as it identifies the main structural components in a system and the relationships between them.
- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.
- Why Software design is crucial?

Fig: The architecture of a packing robot control system



Architectural abstraction

- *Architecture in the small* is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components.

- *Architecture in the large* is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.

Advantages of explicit architecture

- Stakeholder communication
  - Architecture may be used as a focus of discussion by system stakeholders.
- System analysis
  - Means that analysis of whether the system can meet its non-functional requirements is possible.
- Large-scale reuse
  - The architecture may be reusable across a range of systems
  - Product-line architectures may be developed.

Architectural representations

- Simple, informal block diagrams showing **entities** and **relationships** are the most frequently used method for documenting software architectures.
- But these have been criticised because they lack semantics, do not show the types of relationships between entities nor the visible properties of entities in the architecture.
- Depends on the use of architectural models.The requirements for model semantics depends on how the models are used.

Box and line diagrams

- Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- However, useful for communication with stakeholders and for project planning.

Use of architectural models

- As a way of facilitating discussion about the system design
  - A **high-level architectural** view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail.
  - Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.
- As a way of documenting an architecture that has been designed
  - The aim here is to produce a complete system model that shows the different components in a system, their **interfaces** and their **connections**.

Architectural design decisions

- Architectural design is a **creative** process so the process differs depending on the type of system being developed.
- However, a number of common decisions span all design processes and these decisions affect the **non-functional** characteristics of the system.

Issues:

- Is there a **generic application architecture** that can act as a template for the system that is being designed?
- What will be the **fundamental** approach used to structure the system?
- How will the structural components in the system be **decomposed** into sub-components?
- What architectural organization is best for delivering the **non-functional requirements** of the system?
- How will the system be **distributed** across hardware cores or processors?
- What **architectural patterns** or styles might be used?
- What **strategy** will be used to control the operation of the components in the system?
- How should the architecture of the system be **documented**?

Architecture reuse

- Systems in the **same domain** often have similar architectures that reflect domain concepts.
- Application product lines are built around a core architecture with variants that satisfy particular customer requirements.
- The architecture of a system may be designed around one of more architectural **patterns** or '**styles**'.
- These capture the essence of an architecture and can be instantiated in different ways.

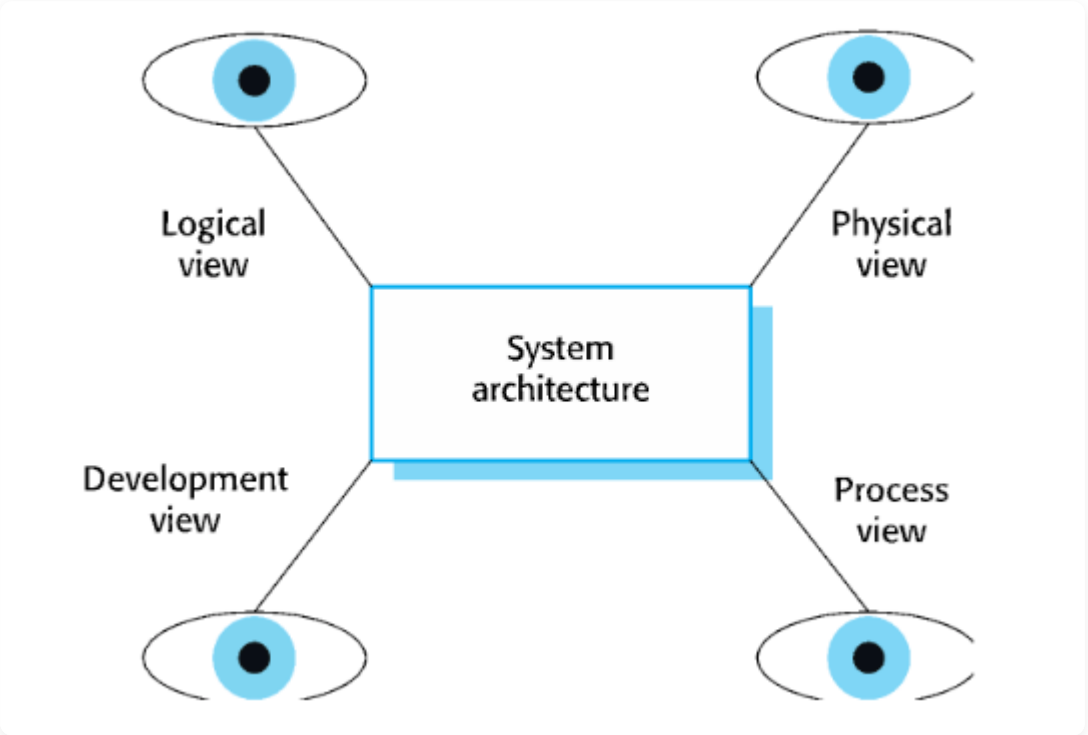
Architecture and system characteristics

- Performance
  - Localise critical operations and minimise communications. Use large rather than fine-grain components.
- Security
  - Use a layered architecture with critical assets in the inner layers.
- Safety
  - Localise safety-critical features in a small number of sub-systems.
- Availability
  - Include redundant components and mechanisms for fault tolerance.
- Maintainability

- Use fine-grain, replaceable components.

Architectural views

- What views or perspectives are useful when designing and documenting a system's architecture?
- What notations should be used for describing architectural models?
- Each architectural model only shows one view or **perspective** of the system.
  - It might show how a system is decomposed into modules, how the run-time processes interact or the different ways in which system components are distributed across a network.
  - For both design and documentation, you usually need to present multiple views of the software architecture.



Views:

- A **logical** view, which shows the key abstractions in the system as objects or object classes.
- A **process** view, which shows how, at run-time, the system is composed of interacting processes.
- A **development** view, which shows how the software is decomposed for development.
- A **physical** view, which shows the system hardware and how software components are distributed across the processors in the system. Related using use cases or scenarios (+1)

Architectural patterns

Architectural patterns:

- Patterns are a means of representing, sharing and reusing knowledge.

- An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.
- Patterns should include information about when they are and when the are not useful.
- Patterns may be represented using tabular and graphical descriptions.

The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.
Example	Figure 6.4 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

Fig: MVC architecture

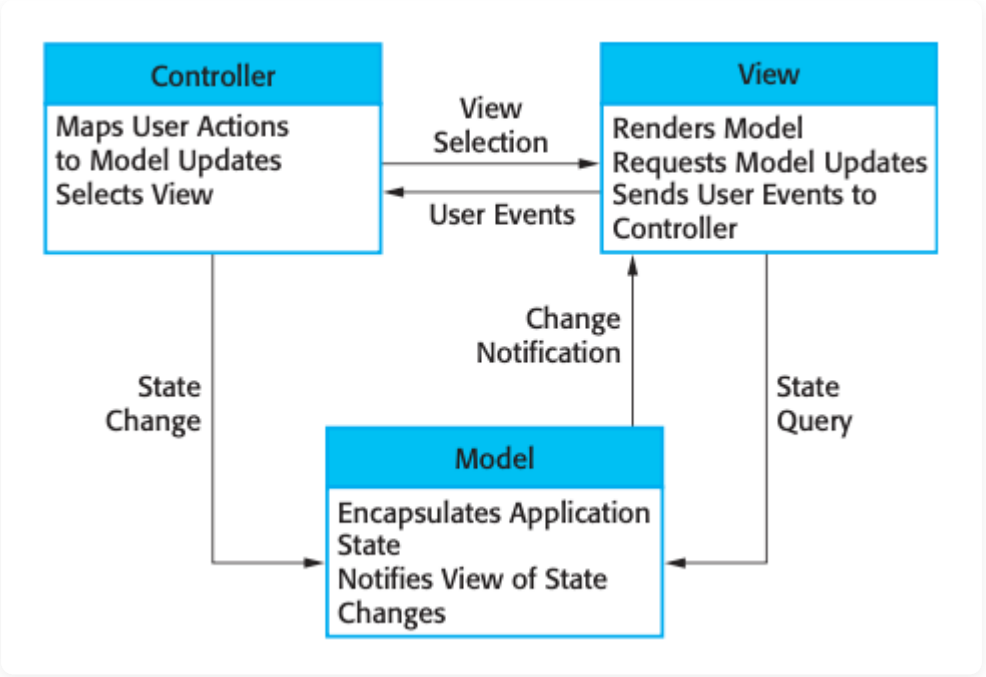
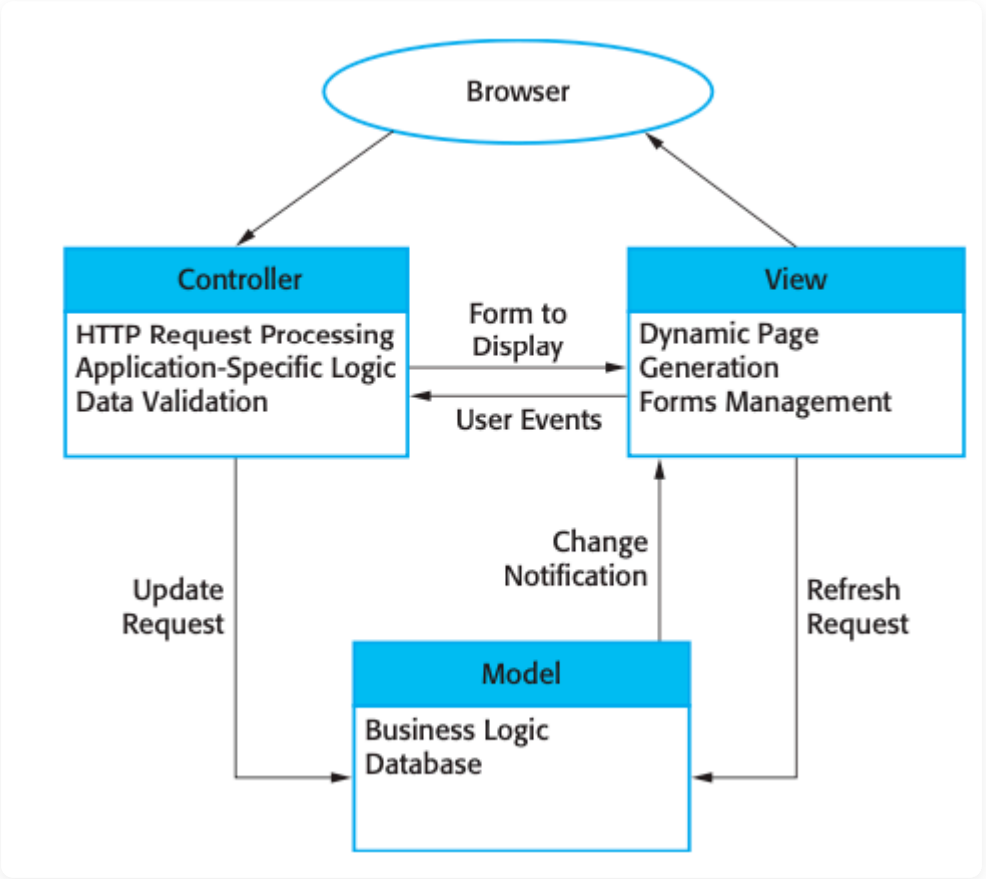


Fig: Web application architecture using the MVC pattern



Layered architecture

Used to model the interfacing of sub-systems.

- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.

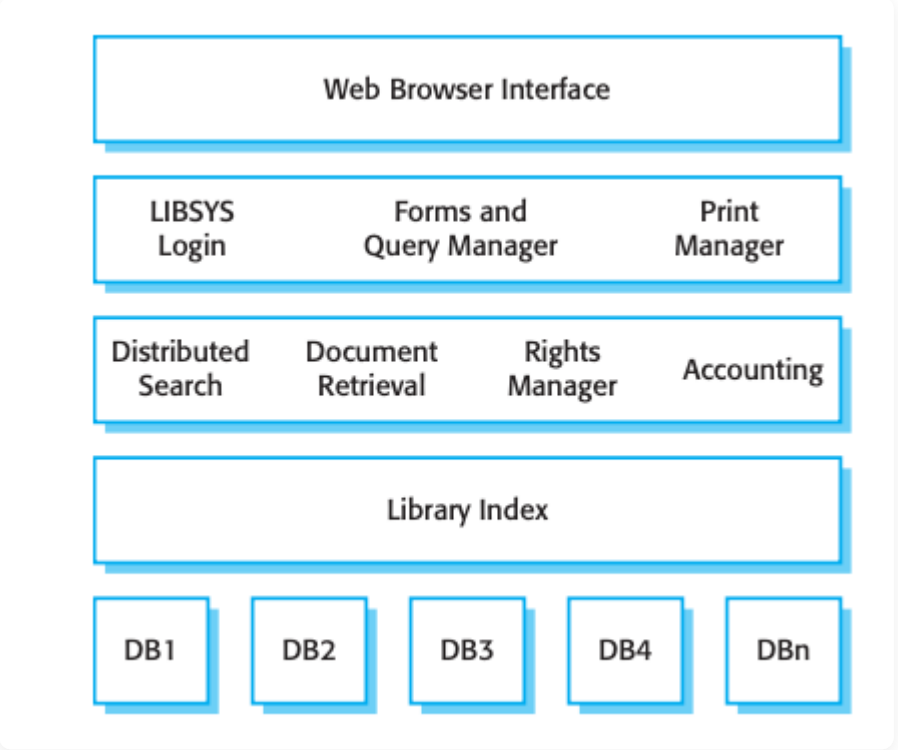
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.

:::success 一個股票交易系統，如何用 MVC 架構來描述？ :::

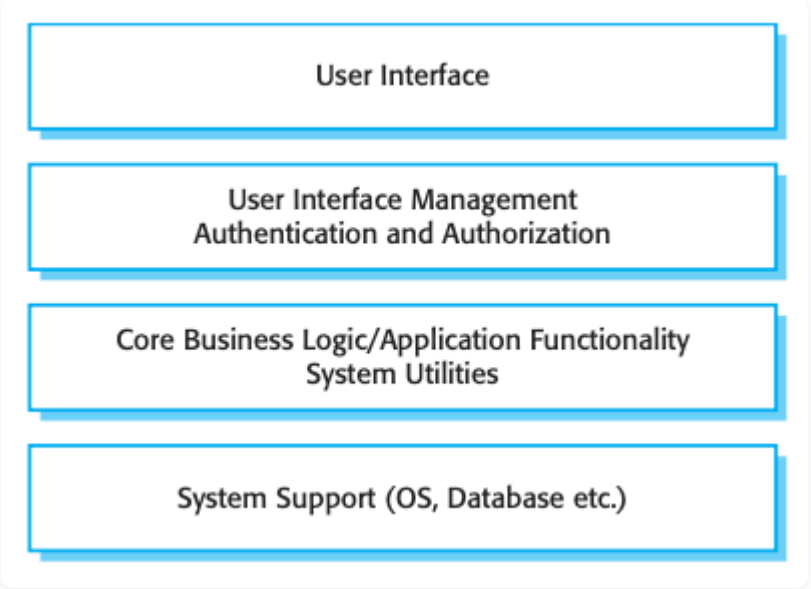
The Layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated **with each layer**. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Fig: The architecture of the LIBSYS system



A generic layered architecture

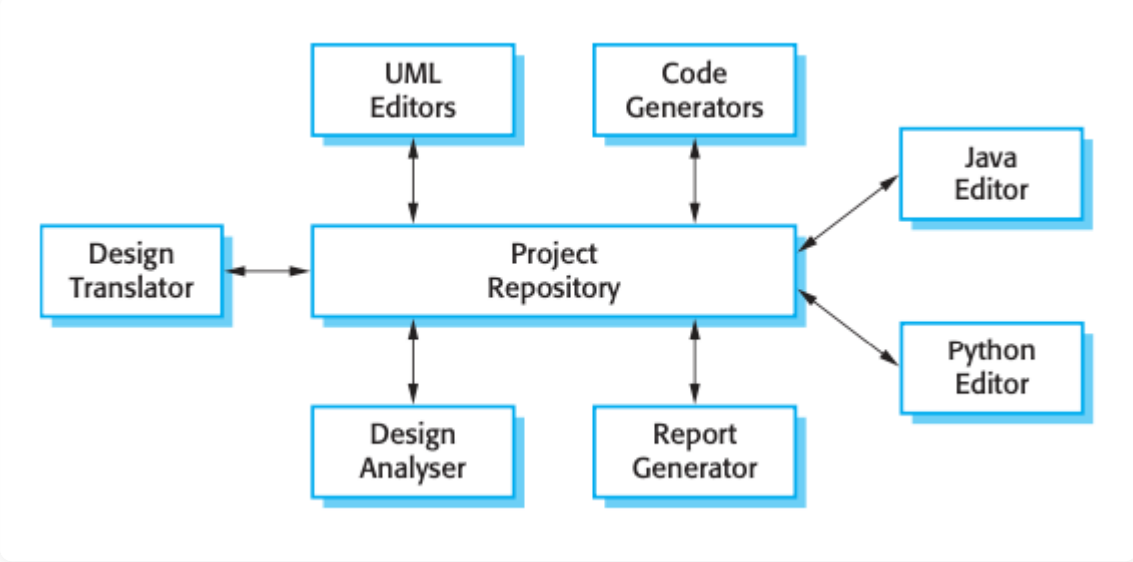


:::success 網球會員管理系統，如何用 layer 架構設計？ :::

Repository architecture

- Sub-systems must exchange data. This may be done in two ways:
- Shared data is held in a central database or repository and may be accessed by all sub-systems;
- Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used a this is an efficient data sharing mechanism.

Fig: A repository architecture for an IDE

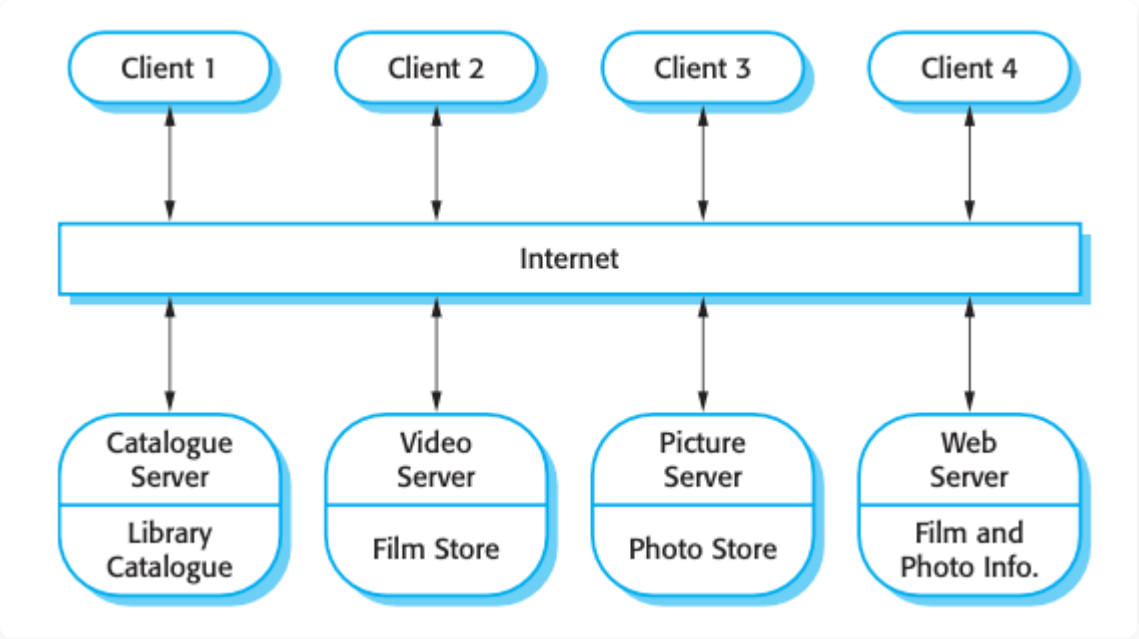


Client-server architecture

- Distributed system model which shows how data and processing is distributed across a range of components.
- Can be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.11 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

Fig: A client— server architecture for a film library

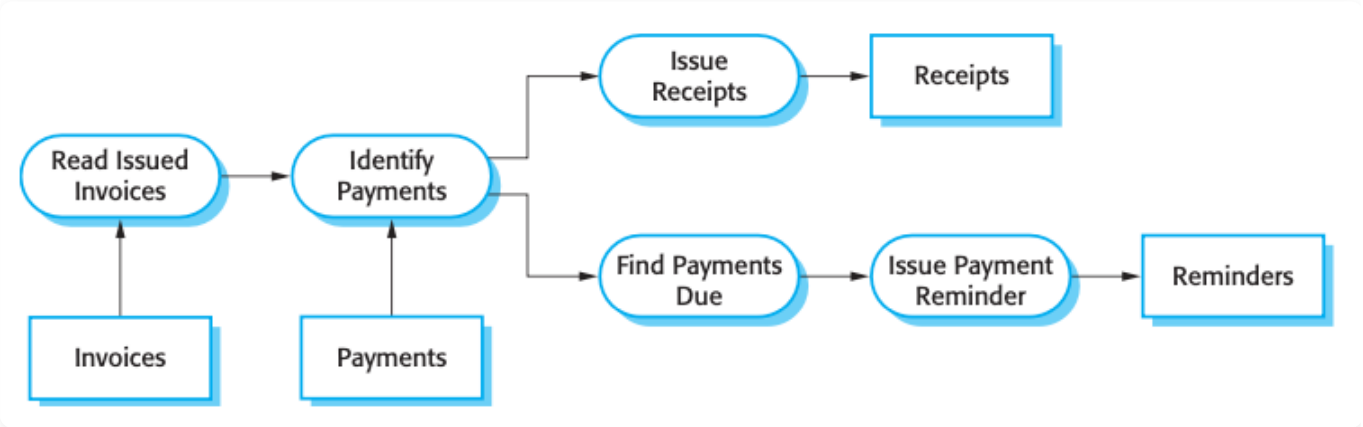


:::success 比較 client server 和 peer-to-peer 的架構 :::

Pipe and filter architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.

Fig: An example of the pipe and filter architecture



Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.13 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

Microservices Architecture (MSA)

Microservices Architecture (MSA) is a method of developing software applications as a suite of small, independent services, each running in its own process. These services are built around specific business capabilities and can be deployed independently.

Think of it as moving from a single, large passenger cruise ship (the **Monolith**) to a fleet of specialized, individual boats.

1. **Small and Focused:** Each service is designed to do **one thing well** (e.g., a "User Service," an "Order Service," or a "Payment Service").
2. **Independent Deployment:** Each microservice can be developed, tested, and deployed **on its own schedule**, without affecting the rest of the application.
3. **Decentralized Data Management:** Services often manage their own **separate databases** to maintain loose coupling.
4. **Communication:** Services talk to each other over a network, typically using **lightweight protocols** like HTTP (REST) or asynchronous messaging (like Kafka).

Core Advantages (Why use it?):

- **Scalability:** You can scale only the services that need more power (e.g., only the "Delivery Tracking Service") instead of scaling the entire application.
- **Resilience:** If one service fails (e.g., the "Notification Service"), the rest of the application can remain operational.
- **Technology Diversity:** Different teams can choose the best programming language or tool for their specific service.

Core Challenges (The Trade-offs):

- **Complexity:** The architecture itself is complex; managing dozens or hundreds of services requires specialized tools for monitoring and networking.
- **Distributed Transactions:** Ensuring data consistency across multiple, separate databases is very difficult.
- **Operational Overhead:** Deploying and managing many small services is much more effort than managing one large application.

:::success Google or Gemini 一下，還有哪些常見的軟體架構？ :::

:::info :basketball: 活動 針對你的專案設計架構- 系統會被分為幾個子系統，每個子系統會負責一些工作。

- 每個成員扮演一個子系統
- 每個成員拿著紙卡，寫在負責的責任、提供的服務（供外界使用或其他子系統呼叫）
- 選一個情境（使用案例），一個成員扮演使用者，開始啟動操作：
  - 當啟動事件開始後，當責的子系統收到後成員講著「收到 x 訊息了，我會做 y，我送了一個請求就給 p 子系統」，並且點一下該系統的負責成員。
  - 重複上述活動，直到請求被完整的處理完。
  - 若遇到遺漏或不清楚的責任與事件，則修改字卡，並重新來過。 :::

Key points

- A software architecture is a description of how a software system is **organized**.
- Architectural design decisions include decisions on the type of **application**, the **distribution** of the system, the architectural **styles** to be used.
- Architectures may be documented from several different **perspectives** or views such as a conceptual view, a logical view, a process view, and a development view.
- Architectural **patterns** are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used and describe its advantages and disadvantages.
- **Models** of application systems architectures help us understand and compare applications, validate application system designs and assess large-scale components for reuse.